Rocket Landing Lab

Technical report and placement preparation handbook

Prepared for Divyansh | 21 October 2026 | Version 3

This project demonstrates deep reinforcement learning through a complete local application: train a rocket policy, intentionally create a reward exploit, evaluate the learned behavior, test wind robustness, and produce automated acceptance evidence. The strongest finding is that optimizing a training reward is not enough: the hover policy earns the incentive it was given yet fails the actual landing objective. Independent evaluation exposes that failure.

The handbook explains the implemented system from first principles to its source files. It is intended for placement preparation: understand the mechanisms, practise explaining decisions, reproduce the demonstrations and distinguish measured achievements from future ideas. The supplied policies use a single training seed. Their results describe those policies, not a universal ranking of reinforcement learning methods.

The delivered application is a CPU-based Python lab with a guided browser interface and the original desktop panel. Version 3 adds SQLite telemetry, analytics and replay, a supervised outcome predictor, local RAG, and randomized-weather PPO fine-tuning. Simulation and analytics need no paid API. Generated assistant explanations use local Ollama with Qwen3 1.7B; evidence retrieval remains available without it. Its original inspiration was Hari (@harixganesh)'s Lunar Lander reward-shaping project; the implementation and upgrades were developed with AI assistance. PPO and the simulator are third-party components, not algorithms invented for this project.

Reading plan

For a first pass, study Chapters 1 to 8 for the learning concepts, Chapters 9 to 16 for implementation and evidence, and Chapters 22 to 26 for interview foundations. Chapters 27 onward cover the web upgrade, new results, demonstration and interview questions. Keep the repository open while reading the source walkthrough. Run a saved policy before trying a new training experiment. On the day before an interview, revise the facts sheet and explain the reward exploit without reading notes.

This report describes the code and saved evidence included in the original project folder and GitHub source. Training code can create additional checkpoints and logs that are intentionally absent from the clean distribution. The report distinguishes existing behavior, inferred interpretation, and proposed extensions.

Contents

1 Project purpose and problem statement 3
2 Technology stack and design choices 4
3 Artificial intelligence and reinforcement learning foundations 5
4 State actions dynamics and the Markov assumption 6
5 The environment loop and episode boundaries 7
6 Neural networks and policy inference 8
7 PPO explained from objective to update 9
8 Advantage estimation optimization and hyperparameters 10
9 Reward shaping and reward exploitation 11
10 Potential shaping and terminal correctness 12
11 Architecture and data flow 13
12 Source walkthrough by module 14
13 Training persistence reproducibility and compatibility 15
14 Evaluation protocol and statistical interpretation 16
15 Original experiment results and analysis 17
16 Weather robustness and the quality gate 18
17 Data files provenance and auditability 19
18 Testing reliability and debugging 20
19 Installation operation and command reference 21
20 Demonstration for placement interviews 22
21 Limitations risks and future work 23
22 Interview questions on fundamentals 24
23 Interview questions on implementation and evidence 25
24 Practical exercises and troubleshooting cases 26
25 Placement narrative resume and revision plan 27
26 Facts sheet and glossary 28
27 Start and demonstrate 29
28 How the pieces connect 30
29 Telemetry: what is actually stored 31
30 Failure predictor: supervised learning without looking into the future 32
31 RAG: retrieval augmented generation 33
32 Broader RL training and honest comparison 34
33 Files, endpoints and reproducibility 35
34 Placement explanation and interview prompts 36
35 Technical deep dive and upgrade interview answers 37
36 Conclusions and references 39

1 Project purpose and problem statement

The problem being solved

A simulated lander must control position, speed and orientation to settle safely. The environment supplies observations and rewards but no labeled correct action for each state. The controller must learn a sequence of actions whose delayed consequences produce a good outcome. Firing the main engine can prevent a crash but also waste reward through fuel penalties; side engines change rotation and lateral movement. A successful policy balances these interacting effects over time.

The engineering problem is broader than making one landing video. We need to know whether the behavior is repeatable, whether the scoring rule represents the task, how the policy behaves when conditions change and whether another person can run the experiment. The project therefore combines training, independent evaluation, saved evidence, a desktop interface and explicit acceptance checks.

Objectives and scope

- Train and retain four PPO policies under different reward rules.
- Demonstrate a repeatable failure caused by rewarding hovering.
- Compare all policies using the original environment reward.
- Measure landing, failure, timeout and engine-use behavior over many episodes.
- Test already-trained policies under three wind settings.
- Make the experiment usable through local controls and command-line commands.
- Preserve configurations, model hashes, raw episode data and readable reports.

The scope is the discrete-action LunarLander-v3 simulator. There is no real rocket hardware, sensor fusion, motor controller, flight certification, multiplayer service or physical fuel measurement. Landing is classified using the simulator's terminal success reward; the implementation does not separately verify that both legs lie inside the flags. A successful resting outcome should therefore not be described as independently certified pad precision.

Deliverables and acceptance

The repository contains runnable source code, tests, four final model ZIP files, configuration and evaluation data, selected recordings and reports. Software acceptance means the checks pass and the workflow executes. Model acceptance means the saved policy satisfies declared statistical thresholds. A rejected model can exist inside fully working software; this distinction is central to the deliberately faulty hover experiment.

2 Technology stack and design choices

Component	Role in this implementation
Python 3.12 on the build machine	Application, wrappers, training and analysis
Gymnasium 1.4.0	Environment interface and LunarLander-v3
Box2D 2.3.10	Two-dimensional physics and collision simulation
Stable-Baselines3 2.9.0	PPO algorithm, policy and rollout utilities
PyTorch 2.13.0 CPU build	Tensors, networks, automatic differentiation and optimizer
NumPy 2.4.4	Numerical operations and statistics
Matplotlib 3.11.1	Learning curves and weather comparison charts
Pillow 12.3.0	Image processing and GIF output
pygame-ce 2.5.8	Simulator rendering through the pygame API
Tkinter and ttk	Native desktop control panel
pytest 9.1.1	Automated behavior and regression tests
SWIG 4.5.1	Build support for Box2D installation

Versions above describe the saved build, not a claim that these are the latest versions. requirements.txt specifies allowed ranges; requirements-lock.txt records tested direct versions. Dependency resolution on a new system can choose different transitive versions. Reproducing an environment means preserving compatible versions, interpreter, platform and seeds, not just copying Python files.

Why these components fit

Gymnasium separates environment dynamics from the learning algorithm. Stable-Baselines3 supplies a maintained PPO implementation, allowing the project to focus on reward design and measurement. PyTorch performs optimization; NumPy is used for calculations that do not need gradients. The observation is only eight values, so a small MLP is appropriate and CPU training is practical. DummyVecEnv adds several streams of experience in one process; it does not create four parallel training machines.

The original Tkinter panel avoids a web server. The new browser interface uses FastAPI and static HTML, CSS and JavaScript to integrate the introduction, analytics and experiment controls. A subprocess performs expensive work while the UI handles events. JSON stores structured settings and summaries; CSV stores rows that can be inspected in a spreadsheet or analyzed in Python. Matplotlib uses the noninteractive Agg backend for file generation. GIF recording uses Pillow and does not require the optional MP4 toolchain.

Standard-library modules also matter: pathlib handles paths, hashlib identifies model bytes, subprocess launches workers, queue transfers progress safely, threading drains process output, datetime names result batches, argparse defines CLI options, and unittest.mock provides a narrow compatibility adapter during model loading.

3 Artificial intelligence and reinforcement learning foundations

Artificial intelligence is the broad goal of building systems that perform intelligent tasks. Machine learning learns behavior or patterns from data. Deep learning uses multilayer neural networks. Reinforcement learning learns a decision policy from interaction and reward. This project sits in their intersection because a neural policy learns sequential control through trial and feedback.

Supervised learning typically receives examples paired with labels, such as an image and the correct class. This project has no table of correct engine choices. The agent generates experience by acting. A reward is feedback about a transition, not necessarily an instruction identifying the best action. An action that loses points immediately can still be useful if it prevents a larger future loss.

Essential vocabulary

Term	Meaning in this project
Agent	PPO's policy and learning procedure
Environment	LunarLander physics, observations and rewards
Observation	Eight numerical values passed to the policy
Action	One of four engine commands
Reward	Scalar feedback after a step
Episode	One attempt from reset to termination or time limit
Policy	Rule mapping observations to action probabilities
Return	Accumulated reward over subsequent steps
Value function	Estimate of expected discounted future return
Rollout	Collected experience used by training or evaluation

The training data is generated online. It changes as the policy changes: a poor controller visits crashing states; an improved one visits approach and touchdown states. This makes data distribution and exploration part of the learning problem. Unlike ordinary random train/test row splits, reinforcement learning evaluation uses new episodes and seeds to measure a policy after learning.

RAG is now implemented as a separate project assistant: it retrieves evidence and uses a local language model to explain it. It does not control the rocket. A supervised classifier separately predicts outcomes from early flight telemetry, while PPO remains the controller. Analytics includes SQLite records, dashboard filters, replay, CSV exports and statistical comparisons. MLOps foundations are present through configuration tracking, repeatable evaluation and quality checks, but a hosted model registry or deployed serving service is not present.

4 State actions dynamics and the Markov assumption

Observation and action spaces

Index	Observation	Practical interpretation
0	Horizontal position x	Left or right relative to the landing reference
1	Vertical position y	Height relative to the reference
2	Horizontal velocity vx	Sideways drift
3	Vertical velocity vy	Ascent or descent
4	Angle	Body tilt in radians
5	Angular velocity	Rate of rotation in scaled simulator units
6	Left contact	Whether the left leg contacts the ground
7	Right contact	Whether the right leg contacts the ground

The actions are 0 for no engine, 1 for the left orientation engine, 2 for the main engine and 3 for the right orientation engine. These are discrete choices, not independent simultaneous throttle sliders. The observation has shape (8,), while the discrete action space has four possible values. The policy receives vectors, not pixels; recording an animation does not turn the model into a computer-vision system.

Position and velocity observations are scaled. Do not interpret every value directly as meters or meters per second. The simulator uses a nominal 50 physics steps per simulated second. A 1,000-step episode limit corresponds to about 20 simulated seconds. Computation without rendering can execute faster than real time. These environment facts are consistent with the installed source and the Gymnasium environment reference [1].

Markov decision process

An MDP consists of states S, actions A, transition dynamics P, reward R and discount factor gamma. The ideal Markov state contains enough information that the next-state distribution depends on the current state and action, not the entire history. The policy uses the provided observation as its state representation. In practice, terrain detail, internal physics and wind phase are not all explicitly included in the eight inputs. The environment's full simulator state is richer than the observation; avoid claiming mathematically complete observability, especially under wind.

The simulator computes forces, motion and contacts after each action. PPO does not learn an explicit physics model here. It learns how to choose actions from experience, making this a model-free learning implementation even though a physics simulator generates that experience.

5 The environment loop and episode boundaries

One attempt step by step

1. Create the environment with the reward preset, render mode and weather parameters.
2. Call reset with an episode seed. Receive the initial observation and info dictionary.
3. Ask the policy to predict an action, or sample an action for a random demonstration.
4. Call step with the action. Receive observation, reward, terminated, truncated and info.
5. Accumulate statistics and continue until terminated or truncated becomes true.
6. Store the episode row, reset for the next attempt and eventually close the environment.

```
obs, info = env.reset(seed=episode_seed)
while True:
    action, _ = model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info = env.step(int(action))
    if terminated or truncated:
        flight = info["flight"]
        break
```

This is a shortened illustration of the real evaluation loop. The production function also records data, optionally renders frames, invokes progress callbacks and closes resources in a finally block. The info dictionary carries diagnostics without increasing the observation vector given to the policy.

Termination versus truncation

Termination indicates an ending defined by the environment, such as failure or coming to rest. Truncation indicates that execution stopped at an external limit, here usually 1,000 steps. The distinction affects learning: there is no continuation after a true terminal state, but a time-limited trajectory can still have continuation value in the underlying task.

The reward wrapper therefore sets next potential to zero only for a true terminal state. For truncation it retains the potential. Stable-Baselines3 handles time-limit value bootstrapping through its vector-environment machinery. If both flags appear, the project classifies the terminal outcome first. Tests explicitly cover this ordering.

An episode can finish between PPO updates. Conversely, one rollout can include several complete episodes plus partial episodes. A training step is not an episode, and a million steps is not a million landing attempts. The reset method also clears all per-episode accumulators so one attempt cannot leak return or potential into the next.

6 Neural networks and policy inference

Network structure

The implementation configures an actor and a critic, each with two hidden layers of 64 units. The actor maps eight inputs through its hidden layers to four action logits. A categorical distribution converts these scores to action probabilities. The critic maps the same observation through its own hidden layers to one scalar value estimate. There is no recurrent memory or convolutional image encoder.

For a dense layer, the computation is z = W x + b followed by an activation. W contains trainable weights and b contains biases. Nonlinear activations allow the model to represent more than a linear rule. With the default MlpPolicy used here, the hidden activation is Tanh. The output logits are not Q-values: they parameterize action selection, while the separate critic estimates state value.

Parameter count

The 8-to-64 layer has 8 x 64 + 64 = 576 parameters. The 64-to-64 layer has 64 x 64 + 64 = 4,160. Each branch therefore has 4,736 hidden-layer parameters. The actor head adds 64 x 4 + 4 = 260, and the value head adds 64 x 1 + 1 = 65. Total actor plus critic parameters are 4,736 + 260 + 4,736 + 65 = 9,797. This small architecture explains why a GPU is not essential for inference.

Training and evaluation actions

Training normally samples actions from the policy distribution, so the agent explores alternatives. Evaluation uses deterministic=True, choosing the categorical mode. The simulator can still differ between episode seeds. Deterministic action selection means the same observation and model give the same chosen action; it does not mean every flight will land or every episode will be identical.

Inference only computes outputs from saved weights. Training additionally stores trajectories, computes advantages, runs backward differentiation and updates parameters. A saved model ZIP contains more than a list of weights: Stable-Baselines3 saves policy state and algorithm metadata, and may include optimizer state. Loading a saved model is not retraining it.

7 PPO explained from objective to update

PPO is an on-policy actor-critic policy-gradient method. It learns from trajectories collected with the current or recently collected policy, then updates that policy for several epochs. This differs from a replay-buffer-based off-policy method such as DQN. PPO is a practical choice here because the project can use its established implementation with a discrete-action MLP; no measured comparison against other algorithms was conducted.

Return value and advantage

The discounted return from time t is G_t = r_t + gamma r_(t+1) + gamma^2 r_(t+2) + ... Gamma = 0.99 reduces the contribution of distant rewards while still assigning value to future outcomes. For intuition, 0.99^100 is approximately 0.366. Training optimizes a discounted objective; reported episode score is an undiscounted sum of original rewards.

The value estimate V(s) predicts expected discounted return from a state. Advantage measures whether an action's outcome was better or worse than that state's expectation. Positive advantage encourages the chosen action; negative advantage discourages it. This is relative credit assignment, not a guarantee that an action is universally good or bad.

Probability ratio and clipping

Define rho = pi_new(a|s) / pi_old(a|s). PPO maximizes the average of min(rho A, clip(rho, 1-epsilon, 1+epsilon) A), where A is the advantage and epsilon is 0.2 in this project. The objective limits the incentive to make an already-favorable probability change excessively large. It is not a hard bound on every neural weight or a proof that the policy cannot degrade [2, 3].

For A = 2 and rho = 1.3, the ordinary term is 2.6 and the clipped term is 2.4. Taking their minimum stops rewarding that extra increase. For negative advantage, the minimum behaves pessimistically in the opposite direction. Explain the sign rather than memorizing only the positive example.

The critic is trained against estimated returns. The implementation uses Stable-Baselines3's optimizer and loss calculation; the project does not implement PPO gradients manually. Advantage normalization and value-loss details should be described as library behavior, while the explicit architectural and training choices live in training.py.

8 Advantage estimation optimization and hyperparameters

Generalized advantage estimation

A one-step temporal-difference residual is delta_t = r_t + gamma V(s_(t+1)) - V(s_t), with terminal handling applied to continuation value. Generalized advantage estimation combines successive residuals: A_t is approximated by delta_t + gamma lambda delta_(t+1) + (gamma lambda)^2 delta_(t+2) + ... Lambda = 0.95 balances short-horizon bootstrapping against longer sampled returns. This trades bias and variance; it is not a reward multiplier.

Configured training values

Setting	Value	Meaning
Requested steps per policy	1,000,000	Target environment transitions
Actual saved steps	1,003,520	Complete rollout boundary
Environments	4	Experience streams in DummyVecEnv
Steps per environment	1,024	Collection before an update
Rollout transitions	4,096	4 x 1,024
Minibatch size	64	Transitions per optimizer minibatch
Epochs	10	Passes over collected rollout data
Learning rate	0.0003	Optimizer step scale
Discount gamma	0.99	Future reward discount
GAE lambda	0.95	Advantage estimator setting
PPO clip range	0.2	Probability-ratio objective clipping
CPU threads	1	PyTorch compute thread setting

One full rollout contains 4,096 / 64 = 64 minibatches. Ten epochs mean up to 640 minibatch optimizer updates per rollout. This is different from 4,096 environment steps. Completing 245 rollouts yields 1,003,520 transitions. Across four policies, training therefore generated 4,014,080 transitions, excluding separate validation and evaluation flights.

The saved model uses the library defaults of normalized advantages, value-loss coefficient 0.5, entropy coefficient 0.0, maximum gradient norm 0.5 and no target-KL early-stop threshold. Even with entropy coefficient zero, sampled actions can explore because the categorical policy is stochastic. Gradient norm clipping limits large gradient magnitudes; it is distinct from PPO probability-ratio clipping. Increasing network size, epochs or threads does not automatically improve quality or wall-clock speed.

The default policy optimizer is Adam. PyTorch's automatic differentiation computes gradients by applying the chain rule backward through the network. Adam maintains running estimates of gradient moments to scale updates. The learning rate controls update scale; it is not the percentage of knowledge learned per step. An optimizer minibatch updates both the policy objective and value estimate according to their weighted loss terms. Gradients are cleared between updates so they do not unintentionally accumulate across minibatches.

9 Reward shaping and reward exploitation

Original reward arithmetic

The installed environment computes a shaping score approximately as -100 times position norm, -100 times velocity norm, -100 times absolute angle, plus 10 for each contacting leg. Ordinary reward is the change in that score since the previous step, less engine-use penalties. If the score improves from -100 to -97 and the main engine fired, reward is 3 - 0.3 = 2.7. Terminal logic replaces the final step reward with +100 for resting or -100 for failure. The episode return sums all steps, so terminal reward and total return are different quantities.

The four controlled experiments

Preset	Added training term	Purpose
baseline	0	Original environment objective
time	-0.03 each step	Penalize delay
hover	+1 on eligible airborne steps	Deliberately demonstrate exploitation
fixed	10 x (0.99 Phi(next) - Phi(current)) - 0.03	Potential shaping plus delay cost

The hover bonus is eligible when height exceeds 0.1, absolute angle is below 0.2, speed norm is below 0.3, neither leg contacts the ground and the step is not terminal. A rocket can collect repeated payments without completing the landing. Ending the episode stops future payments, creating an incentive that conflicts with the intended objective.

Consider only the added bonus and engine cost: at most 1,000 eligible steps would pay 1,000 bonus points, while using the main engine every step costs 300. This is an illustration of the incentive, not an exact observed return, because eligibility, original shaping, episode boundaries and discounting also matter. The actual evidence is the hover model's recorded behavior and high timeout rate.

Reward hacking is optimization of a proxy that does not faithfully represent the desired goal. It does not require malicious intent or awareness. The agent is following the scoring rule. The remedy must change or constrain that rule, and independent evaluation must measure the real objective. A training curve can look good while task success remains poor.

10 Potential shaping and terminal correctness

The potential function is Phi(s) = -(sqrt(x^2+y^2) + 0.5 sqrt(vx^2+vy^2) + 0.5 abs(angle) + 0.1 abs(spin)). Larger potential means a more favorable state under this heuristic. The fixed wrapper adds a discounted difference of potential, scaled by 10, and subtracts the per-step time cost.

A worked transition

Suppose Phi(current) = -0.6 and Phi(next) = -0.5. The progress term is 10 x (0.99 x -0.5 - -0.6) = 1.05. After the time cost, extra reward is 1.02. If the original reward is 2.7, the shaped training reward is 3.72. Evaluation later uses only the original 2.7 for that transition.

The term is not simply Phi(next) - Phi(current), because it uses gamma. If potential stays at -0.5, the progress term is 10 x (0.99 x -0.5 + 0.5) = 0.05; subtracting 0.03 leaves 0.02. Thus it is inaccurate to say every stationary step must have negative shaped bonus. The theoretical role of the discounted potential difference concerns the complete discounted return, not each isolated sign.

Why terminal handling matters

Over a trajectory, discounted potential differences telescope. With proper terminal treatment, the sum contributes a start-state offset rather than arbitrary extra preference over completed paths. The project uses zero potential after true termination and retains potential at time-limit truncation, where continuation value is bootstrapped. Reset initializes previous_potential from the new episode.

Potential-based shaping has a policy-invariance result under specific assumptions, including the appropriate discount and boundary handling [5]. That result does not imply this entire fixed preset is objective-preserving: the additional time penalty deliberately changes the objective. Finite training, function approximation and exploration can also change observed outcomes. The fixed model is an empirical experiment, not a proof of universally optimal control.

The shaping wrapper copies original reward into info before changing the scalar returned to PPO. FlightMetrics can therefore track original and shaped returns simultaneously. This separation prevents the added reward from contaminating the final benchmark.

11 Architecture and data flow

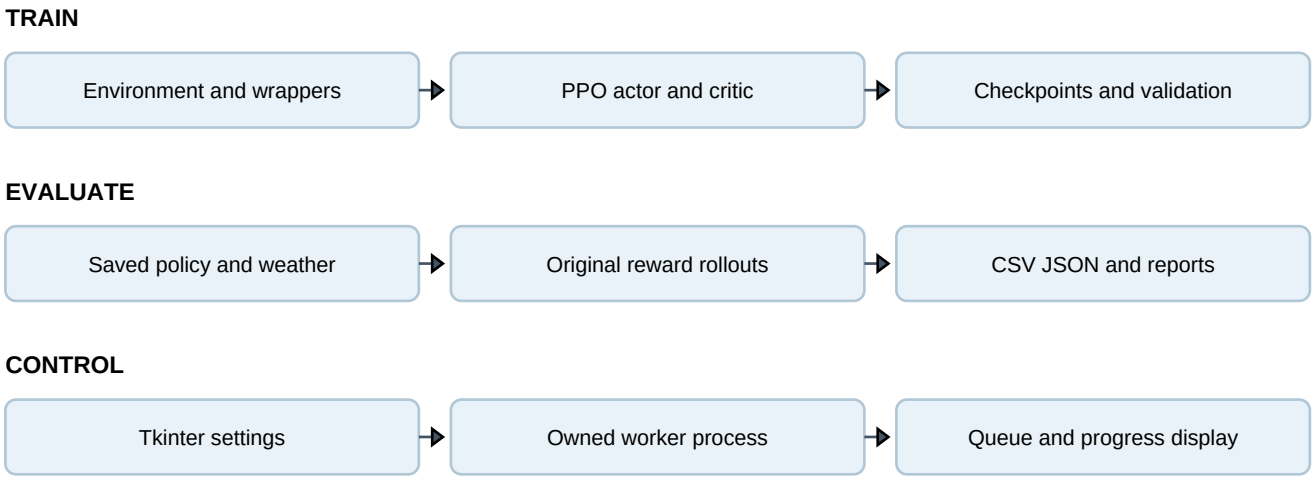


Figure 1 The three execution paths share the same model and evaluation modules.

Training path

The CLI selects a preset and calls training.train. A factory creates LunarLander, optionally wraps it with RewardShaping, then adds FlightMetrics. Monitor captures episode information, and DummyVecEnv presents several environments to PPO. PPO collects experience and updates weights. The Progress callback periodically evaluates the current model on original rewards, writes validation history and saves checkpoints.

Evaluation path

The CLI or desktop worker loads a saved policy with load_ppo. rollout creates an environment with the baseline reward by default, applies the selected weather and executes deterministic predictions. FlightMetrics emits one row at the end of each episode. summarize calculates statistics; report writers persist JSON, CSV and HTML. GIF capture wraps rendering when recording is requested.

Desktop path

ControlPanel gathers settings and calls JobRunner.start with an argument list. JobRunner launches a separate Python process, not a shell string. A background reader thread drains stdout and stderr into a queue. Tkinter polls that queue through window.after, updates progress text and detects process completion. Tkinter widgets are updated on the UI thread rather than the reader thread.

Only one task runs per panel. Start buttons are disabled while it runs, and Stop terminates the process owned by that runner. Closing the panel also closes its active worker. Termination can leave partial outputs; therefore completed manifests and generated final reports are meaningful indicators. The application does not claim transactional rollback of all partially written files.

Separation of responsibilities

The environment defines physics, wrappers shape or measure feedback, PPO learns, evaluation measures fixed policies, and the UI orchestrates. This organization allows the same experiment to run through commands or buttons. It also creates test seams: reward arithmetic can be tested with a scripted environment without running a million-step training job.

12 Source walkthrough by module

Module	Main responsibility	What to explain in an interview
rewards.py	Presets, potential, wrappers, make_env	Reward design and outcome accounting
training.py	PPO creation, callbacks, resume	How experience becomes saved weights
models.py	Checkpoint loading adapter	Narrow compatibility workaround
evaluation.py	Episode loops and summaries	Honest original-reward measurement
recording.py	GIF wrapper	Rendering is separate from observation
scenarios.py	Weather definitions	Explicit experiment settings
robustness.py	Multi-model weather suite	Repeatable comparison and exports
quality.py	Acceptance policy and checks	Model pass/fail versus software success
catalog.py	Model discovery and saved-score matching	Preventing stale score labels
jobs.py	Subprocess lifecycle and queue	Responsive interface and task ownership
panel.py	Tkinter controls	User settings, progress and report navigation
reporting.py	Original experiment report	Aggregation of saved evidence
cli.py	Argument parsing and command dispatch	Reproducible non-GUI interface

Follow one concrete code path

For a calm evaluation, main parses evaluate, loads the saved model, and calls evaluate_model. That function invokes rollout with the scenario. rollout calls make_env with baseline shaping, resets seed + episode index, repeatedly predicts and steps, and reads info['flight'] at the boundary. summarize derives metrics, evaluate_model adds model identity and seed metadata, then writes evaluation.json, evaluation.csv and report.html.

For a quality run, main calls run_gate, which writes acceptance-policy.json before running the suite. run_suite visits every model and condition, writes episode results and updates completed_cases. Only after all cases and report generation succeed does the manifest become complete. audit checks the complete suite, evaluates each model's conditions and rehashes the model file before writing quality-gate.json and quality-gate.html.

The catalog associates a saved score with a checkpoint only when its filename and SHA-256 match. A best checkpoint must not inherit the final checkpoint's evaluation merely because both live in the same directory. The clean distribution includes final models only; newly trained runs can create best and intermediate checkpoints.

13 Training persistence reproducibility and compatibility

Training creates runs/<preset>/seed_<seed> and records the configuration before learning. A new run refuses to reuse an existing config unless resume is requested. Resume verifies the same preset, seed and run directory, then loads the policy with its environment. Continuation metadata is appended instead of silently replacing the original settings.

The progress callback keeps a recent window of episode statistics and prints raw and shaped averages. Approximately every 50,000 environment steps it evaluates ten episodes starting at seed 50,000. It saves a checkpoint and updates best_model when the original validation mean improves. At the end, final_model is saved with total steps and elapsed-session metadata. A keyboard interrupt is caught so an interrupted final checkpoint can still be saved; force-killing a process does not offer the same guarantee.

Why final models matter here

Selecting the best original-reward checkpoint can hide a later reward exploit. The primary experiment therefore compares final models: what did each reward teach after the allotted training? Best-checkpoint selection is a valid different experiment, but it must be named. The project should not switch selection rules after seeing favorable results without disclosing it.

Seeds and reproducibility boundaries

The saved policies use training seed 0. Validation seeds start at 50,000; the original final evaluation uses 100,000 to 100,099. An earlier weather comparison used 200,000 to 200,019. The included quality evaluation uses 300,000 to 300,049. The same numerical evaluation seeds are used for each model and condition, but wind can consume random numbers differently, so this does not promise identical physical starts across weather modes.

Reproducibility also depends on installed versions, platform and deterministic implementation details. A fixed seed controls stochastic choices; it is not a proof of cross-hardware bitwise equality. The current study contains only one training seed per reward. Repeated evaluation seeds quantify episode variation, not variation due to training initialization.

Compatibility fix

The local Torch build had trouble reading an inner ZIP member passed by SB3. load_ppo temporarily wraps torch.load so ZipExtFile inputs become BytesIO. It forwards existing options and restores the original function after the context. The change does not rewrite checkpoint bytes. A test verifies prediction and weight round trips. Because the temporary patch is process-wide, concurrent model loading would need additional design; loading here is single-threaded.

14 Evaluation protocol and statistical interpretation

Metrics

Metric	Calculation or interpretation
Original return	Sum of base rewards over one episode
Mean return	Arithmetic mean of episode returns
Standard deviation	Sample standard deviation using ddof=1
Landing rate	Landed episodes divided by total episodes
Timeout rate	Truncated nonterminal episodes divided by total
Mean steps	Average episode length
Fuel proxy	0.3 x main steps + 0.03 x side steps
Hover fraction	Eligible hover steps divided by episode steps
Wilson interval	Binomial uncertainty interval for landing rate

The original mean-return benchmark is 200 in calm conditions. It is not applied as the same formal task benchmark to windy cases. A high mean can coexist with failures, and high landing rate can coexist with low scores caused by fuel cost or inefficient behavior. Both should be reported. Fuel proxy is not fuel mass and does not represent a finite fuel tank.

Confidence intervals

Let n be evaluated episodes, k be landings, p = k/n and z approximately 1.96. The Wilson center is (p + z^2/(2n)) / (1 + z^2/n). The radius is z times sqrt(p(1-p)/n + z^2/(4n^2)), divided by (1 + z^2/n). The interval is center minus radius to center plus radius, bounded to [0,1].

Wilson intervals behave sensibly at zero and perfect success, unlike a simple symmetric normal interval that can collapse or exceed probability bounds. Zero successes in 20 trials does not prove the true success probability is zero. The interpretation assumes the episode sampling reasonably supports the binomial model. It does not cover training variation, simulator defects or arbitrary deployment changes.

The quality gate uses the lower endpoint of a two-sided 95% interval as a conservative criterion. It is not a simultaneous 95% guarantee across all models and scenarios; no multiple-comparison correction is implemented. A confidence interval is not the probability that this one fixed interval contains a random true parameter. Its confidence level describes long-run coverage of the construction under its assumptions.

15 Original experiment results and analysis

The table records 100 evaluation episodes per saved policy using original rewards, deterministic actions and calm weather. All four models trained for 1,003,520 steps with seed 0. The test episodes start at seed 100,000.

Policy	Mean score	Std dev	Landed	Timeout
baseline	238.80	65.05	94%	0%
time	235.91	39.46	89%	11%
hover	-8.24	20.44	0%	100%
fixed	221.27	41.85	92%	7%



Figure 2 Original validation scores and shaped training curves from the saved experiment.

What the evidence supports

Baseline produced a mean original score of about 238.8 and 94 landings out of 100. The time-penalty and fixed policies also exceeded the 200 mean benchmark. The hover policy had a mean near -8.2, zero landings and 100 timeouts. Its failure is therefore visible in outcomes, not merely a small difference in score.

The fixed policy does not exceed baseline on every original calm metric. Its mean is about 221.3 and its landing rate is 92%. The name fixed refers to fixing the repeated-hover incentive design, not to an established claim that the policy is best in every setting. A realistic discussion includes the baseline's stronger calm score.

The saved learning chart compares validation on original rewards with smoothed shaped training rewards. Different reward scales make shaped curves unsuitable for ranking task performance across presets. A positive training trend can reflect learning an exploit. The plot is retained as historical evidence; some full training monitor logs were excluded from the upload copy, so reproducing every plotted training point requires retaining the original working project or rerunning training.

What the evidence does not establish

There are no multiple independent training seeds, algorithm comparisons, random-policy confidence study, wall-clock benchmarking across machines or real-world flights. The table cannot establish statistical superiority of one reward method in general. It does show a clear objective mismatch for this hover policy and successful original-task behavior for the other saved policies.

16 Weather robustness and the quality gate

Test matrix

Calm disables wind. Breezy uses wind power 8 and turbulence power 0.5. Gusty uses 15 and 1.5. These are simulator magnitudes, not wind speeds in kilometers per hour. All policies were trained in calm weather; the stress test changes the environment during evaluation without updating weights.

The included quality suite has four policies, three conditions and 50 episodes per case, for 600 total flights. Episode seeds are 300,000 to 300,049. The acceptance criteria were set before that run: at least 50 episodes per condition; landing Wilson lower bounds at least 0.70 calm, 0.60 breezy and 0.50 gusty; timeout rate at most 0.20 everywhere; calm mean score at least 200; matching saved model SHA-256.

Policy	Calm landed	Breezy landed	Gusty landed	Gate
baseline	84%	62%	30%	FAIL
fixed	86%	92%	82%	PASS
hover	0%	0%	4%	FAIL
time	94%	94%	62%	FAIL

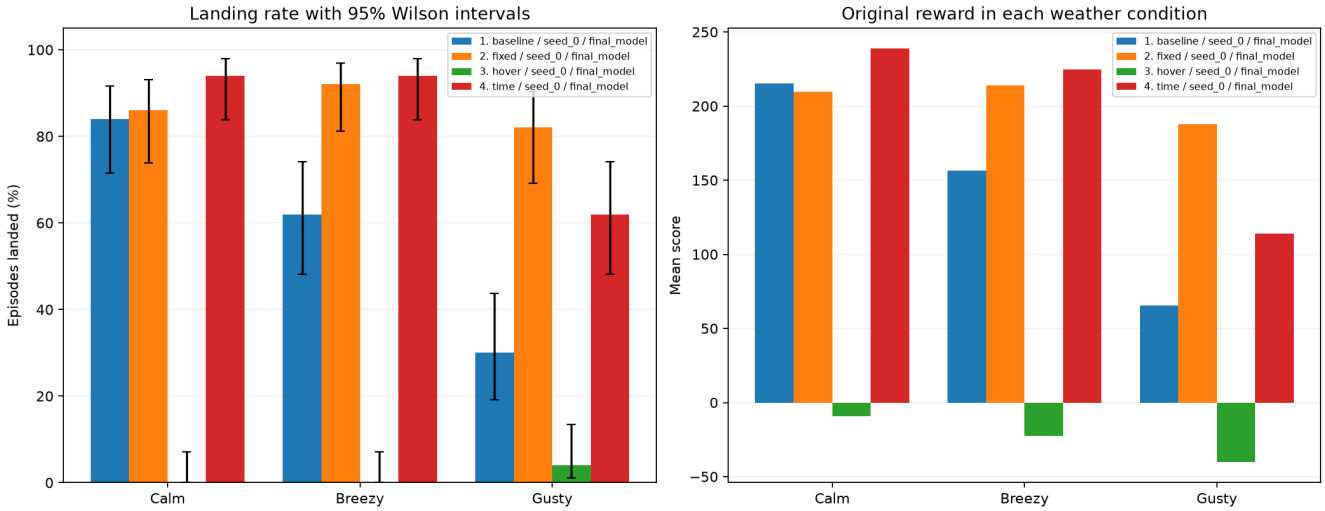


Figure 3 The 600-flight weather comparison with landing uncertainty intervals and original scores.

Reading the pass and fail decisions

The fixed policy passed all checks in this suite. Baseline failed breezy and gusty reliability. Hover failed reliability and timeouts, as well as the calm score. The time policy landed in 31 of 50 gusty episodes, an observed 62%, but its Wilson lower bound is approximately 48.15%, below the 50% gate. This is a concrete example of requiring evidence strength rather than accepting the point estimate alone.

The aggregate gate fails if any included model fails, so a suite intentionally including hover is expected to fail overall. A completed software run still writes a report. By default that returns normal success to indicate report generation completed; --fail-on-rejection requests exit code 2 for model rejection, making the command usable in future CI automation. No hosted CI pipeline or automatic deployment is configured.

The thresholds are project acceptance choices, not universal industry standards. The gate checks complete manifests, one result per required condition, sample size and identity. It does not digitally sign evidence or defend against a malicious person editing both the results and metadata. Hashing model bytes ties results to a file; it is not full provenance security. The clean copy retains combined raw data and canonical summaries while redundant per-condition exports are removed.

17 Data files provenance and auditability

Why multiple formats exist

config.json records training settings and selected package versions. training.json records the saved training step count and interruption status. evaluation.csv contains one row per original test episode; evaluation.json summarizes that test and names the model hash. validation.jsonl is an append-friendly history of checkpoint validation. These have different purposes and should not all be dismissed as unnecessary text files.

For a weather suite, manifest.json records models, conditions, first seed, reward protocol, status and completed-case count. results.json is the structured case summary. comparison.csv is its flat spreadsheet-friendly view. episodes.csv combines individual flight rows across every model and weather condition. quality-gate.json records acceptance results and policy thresholds; the HTML files present charts and decisions for humans.

Each checkpoint's SHA-256 is computed from its bytes. If those bytes change, the digest should change with overwhelming probability. The catalog and gate compare digests to avoid attaching an old result to a modified policy. This is more reliable than relying only on a filename such as final_model.zip, which could be overwritten.

File lifecycle

Training and testing create new timestamped results so repeated runs do not silently overwrite earlier evidence. Interrupted suites remain distinguishable from completed suites. The clean repository keeps selected evidence; .gitignore excludes environments, caches, temporary runs and newly generated files unless intentionally included. The included examples must remain explicitly allowed by ignore rules.

Raw data makes claims inspectable. For example, a reviewer can count outcomes in episodes.csv, recompute the landing rate and compare the reported score. The quality auditor currently consumes saved summaries; it does not automatically recompute every summary from CSV during audit. Independent recomputation is therefore a useful additional review, and a future strengthening could validate summary-to-episode consistency inside the audit itself.

Do not publish model scores without the conditions, sample count, seed protocol and checkpoint identity. A result such as 94% means little unless the reader knows it came from 100 calm episodes on one particular final baseline policy.

18 Testing reliability and debugging

The updated automated suite contains 41 passing cases, including eight new web-lab data and backend tests. It tests important behavior rather than claiming that every possible UI path or physics state has been exhaustively explored. Passing software tests does not mean every trained model passes the quality gate.

Area	Example checks
Reward arithmetic	Hover eligibility, fast descent exclusion, correct added terms
Episode boundaries	Terminal versus truncation potential, reset state, outcome ordering
Physics preservation	Same actions and seed give identical observations with or without shaping
Evaluation	Repeatable seeds, original return retained, empty input rejected
Statistics	Wilson intervals at zero, perfect and ordinary success rates
Models	Save/load preserves predictions and weights
Recording	Real multi-frame GIF output
Weather	Correct settings reach simulator and seeded behavior repeats
Catalog	Final/best association and stale hash rejection
Jobs	Output streaming and refusal of concurrent starts
Quality	Missing, duplicate, insufficient, weak and nonfinite evidence rejection
Integrity	Changed model weights fail; incomplete manifest is rejected

The upload-copy integration check additionally loaded all four included models, flew each in all three conditions, recorded and decoded a GIF, and drove a hidden control panel through a real background evaluation. It verified report/media links and matched all included 600-flight model hashes. Tests used the installed dependencies on the development computer; a clean-room installation on an unrelated computer has not been validated.

Debug systematically

If a model fails, first distinguish an execution problem from a behavior problem. Import errors or a missing file are software issues. A valid flight that crashes is a policy outcome. A high shaped score with poor original return suggests reward misalignment. A good calm policy that degrades in wind suggests distribution shift. A score that no longer matches the UI may indicate a changed checkpoint or wrong scenario metadata.

For dependency problems, read the first meaningful exception rather than only the final process exit. For Box2D build errors mentioning SWIG, setup.ps1 installs SWIG before the requirements. The known Torch inner-ZIP issue is handled by the loading adapter. Third-party SWIG deprecation warnings seen in tests are warnings, not test failures.

19 Installation operation and command reference

First use on Windows

Install a compatible Python, open PowerShell in the repository root and run the setup script below. It creates a local virtual environment, upgrades pip, installs SWIG, installs requirements and runs tests. Package installation needs internet access. After setup, the included policies can run without a new training session. The original project folder uses .gitignore to exclude .venv because environments are machine-specific and large.

```
powershell -ExecutionPolicy Bypass -File .\setup.ps1
.\.venv\Scripts\python.exe -m rocket_landing panel
```

Open-Web-Lab.cmd is the recommended entry point. It starts a loopback server on an available port and opens the introduction. Keep its console open while using the app. Generated assistant answers require Ollama with qwen3:1.7b installed.

The desktop launchers are convenience wrappers. Open-Control-Panel.cmd starts the app. Watch-Landing.cmd and Watch-Hover.cmd play saved policies. View-Results.cmd opens original results. View-Wind-Results.cmd opens the retained quality suite's weather comparison. View-Quality-Results.cmd opens the acceptance report. Run-Quality-Check.cmd executes the standard suite.

Useful commands

```
# Explore the interface
.\.venv\Scripts\python.exe -m rocket_landing --help
# Train a new seed without replacing delivered seed zero
.\.venv\Scripts\python.exe -m rocket_landing train --preset fixed --seed 1 --steps 1000000
# Evaluate original reward on calm episodes
.\.venv\Scripts\python.exe -m rocket_landing evaluate --model `
runs/fixed/seed_1/final_model.zip --episodes 100
# Watch a saved policy with wind
.\.venv\Scripts\python.exe -m rocket_landing watch --model runs/fixed/seed_0/final_model.zip `
--scenario gusty --human
# Quality check for a selected model and new test seeds
.\.venv\Scripts\python.exe -m rocket_landing quality --models `
runs/fixed/seed_0/final_model.zip --seed 400000 --episodes 50 --fail-on-rejection
# Run automated checks
.\.venv\Scripts\python.exe -m pytest -q
```

Commands run from the project root so relative evidence paths resolve. New generated artifacts should be reviewed before adding them to Git. The original calm evaluation command defaults to the model's directory; choose an explicit --output when preserving a previous evaluation is important. The GUI creates separate result directories for its evaluation jobs.

Recording requests rgb_array rendering and captures actual simulator frames. GifRecorder keeps approximately every second physics frame, resizes to 400 by 267 pixels, uses a 128-color palette and saves frames with a 40 millisecond duration. This gives approximately 25 frames per second while limiting file size. Terminal frames are also captured, so the final sampling interval can differ slightly. Frames are buffered for an episode and saved at its boundary or close. Human rendering opens a live game window; it is separate from headless training and from file recording.

20 Demonstration for placement interviews

Prepare before the session

Run setup and open the control panel ahead of time. Confirm the four final policies appear with their saved scores. Open the original and quality reports, and keep the saved GIFs available as a fallback. Use a charged laptop and allow the native simulator window to open. Do not start million-step training during a short interview.

Original control-and-reward demonstration

1. In the first 30 seconds, explain the problem: learning engine actions from state observations and rewards, then verifying whether the reward teaches the desired behavior.

2. Show baseline in calm weather for roughly one minute. Point out position, descent and orientation control. Explain that the network receives eight numbers, not the screen image.

3. Show hover for one minute. State the exact added incentive and explain why ending the episode can be worse than continuing to collect points.

4. Show fixed for one minute. Explain the potential difference and time penalty. Do not claim it must win every flight.

5. Open original results for one minute. Compare original mean return, landing and timeout rates. Show why the hover training objective cannot serve as the final score.

6. Open the weather chart for one minute. Discuss distribution shift and the fixed model's measured performance under the included test protocol.

7. Open the quality report for the remaining time. Explain the time policy's 62% gusty point estimate and 48.15% confidence lower bound, then show that a failure is deliberately reported.

If asked to prove implementation knowledge, open rewards.py and trace reset, step, original reward storage and terminal potential. Then open evaluation.py and explain how it removes custom shaping without changing the already-learned policy weights. This is more persuasive than showing a long list of dependencies.

A sixty second explanation

I built a local reinforcement learning lab around LunarLander. PPO learns four policies from numerical observations using different reward rules. One rule deliberately rewards hovering, which exposes how a high training objective can conflict with landing. I evaluate saved models using original rewards, compare calm and windy conditions, and use a quality gate with sample-size, confidence, timeout and model-identity checks. The baseline landed in 94 of 100 original calm test episodes, while the hover policy timed out in all 100. In a separate 600-flight weather suite, the fixed-reward policy passed every declared project criterion. The evidence is simulation-only and uses one training seed per policy.

21 Limitations risks and future work

Current limitations

One training seed per reward is the largest experimental limitation. Evaluation episodes vary, but the learned weights are still one sample from the training process. The observed results cannot establish broad superiority. The existing stress tests cover only three fixed simulator settings, not a continuous range of winds, gravity changes, sensor noise or actuator failures.

The four original policies were trained on calm dynamics and tested under wind without adaptation. Version 3 also fine-tunes a copy of the fixed policy under randomized weather; its separate comparison is discussed later. The observation omits some internal simulator information. There is no learned dynamics model, recurrent memory, uncertainty-aware controller or fallback safety controller. Engine-use cost is only a proxy, and the success flag does not independently certify within-pad accuracy.

The software is local and intended for trusted project artifacts. Model-loading adaptation does not make arbitrary downloaded checkpoint archives safe. The gate does not cryptographically sign data, verify every raw CSV aggregate or maintain a tamper-proof registry. Stopping a job may leave partial output. The temporary Torch patch assumes single-threaded loading.

Proposed next experiments

First, train at least several independent seeds per reward and compare aggregate results under a prespecified protocol. Second, reserve new evaluation seeds after development; repeatedly tuning to the same test set turns it into validation data. Third, isolate the fixed reward's components with ablations: progress only, time only, both and baseline. Fourth, extend the implemented randomized-weather experiment with controlled weather/reward ablations and multiple independent training seeds.

Potential engineering extensions include an automatic summary-to-raw-data audit, environment/package snapshots, resource measurements, explicit pad-position success criteria, a CI workflow, a model registry and richer failure classification. These are future work, not features already delivered. Hosting CI or adding a registry would require infrastructure choices; their names alone do not make the present project production-ready.

Avoid changing thresholds simply until a favored model passes. Declare the acceptance rationale, keep the original failed results, and use fresh held-out evaluation after making development changes. The quality of the engineering argument depends as much on transparent failure handling as on a successful video.

22 Interview questions on fundamentals

Why reinforcement learning instead of supervised learning

There are no supplied correct engine-action labels. The agent learns from interaction, delayed consequences and scalar feedback. Supervised imitation would be another possible approach if expert trajectories existed, but it would solve a different data-availability problem.

What exactly is learned

The trainable neural-network parameters of the actor and critic. The actor maps observations to action probabilities; the critic estimates future return. The environment's Box2D physics is provided and is not learned by PPO in this implementation.

Why is this deep learning when the network is small

It uses multiple learned nonlinear layers and backpropagation. Deep learning does not require a billion-parameter model. Here a small network is appropriate because inputs are eight structured values rather than high-resolution images.

What is the difference between reward return and value

Reward is immediate feedback from one step. Return aggregates future rewards, often with discounting during learning. Value is the model's expectation of that future return. Reported episode score is a realized undiscounted original return, not the critic's prediction.

What is a policy and is it deterministic

A policy maps a state or observation to an action distribution. Training samples from the categorical distribution. The evaluation helper requests deterministic prediction, selecting its mode. Environment variation can still change trajectories and outcomes.

What does on policy mean here

PPO updates from data collected by a recent version of itself, with stored old action probabilities used in the objective. It reuses that rollout for several epochs. It does not use an indefinitely growing replay buffer of old experiences as the main training design.

Is PPO clipping the neural weights

No. Its clipped objective uses the ratio of new and old action probabilities for sampled actions. Gradient-norm clipping is a separate optimizer stabilization mechanism. Neither makes performance improvement a guarantee.

Why do we need a critic

It estimates the expected outcome from a state, supporting advantage estimation and reducing variance in the policy update. The critic is trained alongside the actor, but deterministic action inference uses the actor's action distribution.

Why not use a CNN RNN or transformer

The input is a compact numerical vector, so an MLP is a simple baseline. A CNN would make more sense for image observations. An RNN could address partial observability or history dependence. A transformer is not justified merely because it is popular; added complexity needs evidence.

Is this RAG or generative AI

The controller uses deep reinforcement learning. The separate assistant uses TF-IDF retrieval and a local language model; it explains sources but never selects engine actions. The outcome classifier uses supervised learning. These are three distinct systems with different input/output contracts.

23 Interview questions on implementation and evidence

How did the reward exploit happen

The hover wrapper pays repeatedly for a slow upright airborne state. Landing ends future bonus collection. The trained hover policy timed out in every original 100-episode calm evaluation, showing that optimizing the proxy did not solve the intended task.

Does fixed shaping guarantee the best policy

No. The potential term has theoretical properties under assumptions, but the time penalty changes the objective and finite PPO training is approximate. The saved fixed policy passed the declared weather checks, while baseline had a higher original calm mean. Both facts should be stated.

How did you avoid rewarding your own evaluation

Evaluation creates the environment with baseline reward, irrespective of the policy's training preset. It keeps the learned weights but removes the wrapper's custom bonus. FlightMetrics also stores base reward separately during shaped training for diagnosis.

What is the most important boundary bug to avoid

Treating truncation as the same as a true terminal state. That can remove continuation value incorrectly. This wrapper uses zero next potential only when terminated, retains it for time limits and resets its previous potential on a new episode.

Why are there four environments if the machine uses one CPU thread

DummyVecEnv manages multiple environment streams sequentially in one process. It supplies diverse episode trajectories per collection cycle. It is not subprocess parallel simulation. The one-thread choice controls PyTorch compute overhead for this small network.

How is the UI kept responsive

Long work runs in a child Python process. A reader thread places output into a queue. Tkinter polls that queue on its event loop. This avoids training or simulation directly blocking widget updates and keeps thread-unsafe widget access out of the reader thread.

Why keep SHA 256 hashes

Names can remain the same while bytes change. A digest identifies the evaluated model bytes and allows the catalog and acceptance audit to reject stale associations. It does not prove the data was honestly produced or protect against coordinated edits to results and hashes.

Why did 62 percent gusty success fail

That estimate came from only 50 trials. The Wilson lower endpoint was about 48.15%, below the declared 50% criterion. The gate requires enough confidence, not just a favorable sample proportion. More evidence could change the interval, but it is not guaranteed to cause a pass.

What would you change with more time

Run multiple training seeds, improve experimental design with ablations and unseen winds, add strict raw-data consistency checks and measure runtime resources. Explain why each addresses a demonstrated limitation instead of promising unrelated features.

What part did you implement versus use from libraries

The application combines custom reward wrappers, metric tracking, training orchestration, evaluation/reporting, weather scenarios, GUI jobs and acceptance checks. Stable-Baselines3 supplies PPO and Gymnasium/Box2D supplies simulation. Be transparent about AI-assisted implementation and demonstrate understanding through changes you can explain and test.

24 Practical exercises and troubleshooting cases

Exercise one trace a reward

Given original reward 2.7, Phi before -0.6 and after -0.5, compute the fixed shaped reward. Answer: the progress term is 1.05, extra reward after time cost is 1.02, and final shaped reward is 3.72. Explain why the evaluation score still uses 2.7.

Exercise two count the rollout

With four environments and 1,024 steps each, how many transitions are collected before an update? Answer: 4,096. With batch size 64 and ten epochs, there are up to 640 minibatch updates for that rollout. Explain why these counts are not episode counts.

Exercise three diagnose a misleading chart

The shaped training mean rises but original evaluation remains low and timeout rate increases. The likely issue is objective mismatch, especially if recordings show repeated hovering. Check base versus shaped rewards, eligibility conditions and episode endings before changing the optimizer.

Exercise four diagnose wind failure

The same model loads successfully and lands in calm but crashes in gusty weather. First inspect scenario metadata and repeatability. If execution is valid, this is behavioral robustness failure, not evidence of a broken model file. Propose wind-randomized training and a held-out robustness test.

Exercise five handle a changed checkpoint

A user replaces final_model.zip but leaves evaluation.json. The catalog should refuse to attach the stale summary when the digest no longer matches. Re-evaluate the new model and save results rather than editing the old score by hand.

Exercise six debug setup

A Box2D build mentions SWIG. Confirm setup.ps1 installed SWIG and that the virtual environment's Scripts directory is on the build path. A different missing dependency should be diagnosed from its own traceback. Do not reinstall every package blindly or weaken security settings globally to fix an unrelated import.

Exercise seven design an honest next comparison

Choose multiple independent training seeds and declare the training budget, checkpoint selection, evaluation seeds, conditions and metrics before the run. Preserve failed runs. Use uncertainty across training seeds as well as episode variation. Do not compare a best checkpoint for one method with an arbitrary final checkpoint for another without explaining the selection difference.

Exercise eight explain a stopped job

Stop terminates the panel's worker. Partial files may remain and should not be read as a completed suite. Inspect the manifest status and completed-case count. A future improvement could write outputs atomically and add an explicit cancelled state, but that is not the current implementation contract.

25 Placement narrative resume and revision plan

How to present the project honestly

Emphasize the complete experimental workflow, not only the animation. The interesting engineering decisions are the deliberately exploitable objective, unshaped evaluation, terminal handling, reproducibility, responsive execution and transparent rejection criteria. State exactly what the measurements represent and acknowledge library contributions and AI assistance when asked.

Suggested resume wording: Built a local AI experiment lab using PPO, Gymnasium, FastAPI and SQLite, with reward experiments, a guided web interface, telemetry analytics, supervised outcome prediction, local RAG and weather-randomized fine-tuning. Evaluated saved policies on original rewards and demonstrated a hovering reward exploit; the baseline landed in 94 of 100 calm test episodes, while a separate 600-flight suite measured robustness across three conditions.

Use this only if you can explain and reproduce the workflow. Avoid claiming a 94% real-world success rate, production deployment, invention of PPO, multi-seed algorithm superiority or a trained large language model. These are not supported by the project.

Seven day preparation plan

Day	Focus	Evidence that you understand it
1	Problem and environment	Explain all eight inputs and four actions without notes
2	RL and PPO	Distinguish reward, return, value, advantage and clipping
3	Reward code	Calculate the example and trace terminal handling
4	Training and persistence	Explain rollout size, checkpoints, seeds and resume
5	Metrics and gate	Explain why 62% can fail a 50% lower-bound gate
6	Web lab, supervised ML and RAG	Explain seed-group splitting, failure recall and retrieved evidence
7	Mock interview	Give a 60-second pitch, answer follow-ups and admit limits

Questions to ask yourself before the interview

Can I identify where the raw reward is stored? Can I change a reward coefficient and predict the possible incentive? Can I explain why more episodes and more training seeds answer different questions? Can I distinguish software failures from model failures? Can I point to the data behind a claimed result? Can I reproduce a command from the repository root without relying on a screenshot?

Prepare a short demonstration and a backup with saved clips. A strong answer states the decision, the reason, the measurement and the limitation. If you do not know an internal library detail, say so and identify the source you would inspect instead of inventing an explanation.

26 Facts sheet and glossary

Facts to remember

- Environment: LunarLander-v3, eight numerical observations, four discrete actions, 1,000-step cap.
- Algorithm: PPO actor-critic, two 64-unit hidden layers per branch, 9,797 parameters in total.
- Training: four sequential vector environments, 4,096 transitions per rollout, 64 batch size, ten epochs, 0.0003 learning rate.
- Discount and GAE: gamma 0.99, lambda 0.95, PPO clip 0.2.
- Saved training: 1,003,520 steps per model, four models, one training seed each.
- Original test: 100 calm episodes per model; baseline 238.8 mean and 94% landed; hover 0% landed and 100% timed out.
- Quality test: 600 flights total; fixed passed all project criteria; the other three failed at least one.
- Evidence: original-reward evaluation, model SHA-256, seeds, CSV rows, JSON summaries and HTML reports.
- Verification: 41 automated tests passed after the upgrade; browser navigation, filtering, replay, assistant answers and the predictor training control were exercised.

Terms in plain language

An ablation removes or changes one component to isolate its contribution. Bootstrapping uses an estimated future value as part of a learning target. Distribution shift means evaluation conditions differ from training conditions. Generalization is useful behavior in those changed or unseen conditions. A checkpoint is a saved training state. An epoch is one pass over the current training batch, not a complete simulated flight.

A heuristic is a designed rule, such as the potential function, rather than a learned fact about optimal behavior. A hyperparameter is chosen outside gradient learning, such as the learning rate. A parameter is learned, such as a network weight. Inference applies trained parameters. A seed initializes a pseudorandom sequence. A proxy metric is an indirect measurement of the desired outcome. Reward exploitation happens when optimizing that proxy produces unwanted behavior.

Reproducibility means another run can follow a specified protocol and compare results meaningfully. Auditability means the evidence and settings can be inspected. Robustness means performance under specified changes, not immunity to every disturbance. Confidence intervals express uncertainty under a statistical model. A quality gate applies explicit acceptance rules; it is only as appropriate as those rules and the evidence supplied.

27 Start and demonstrate

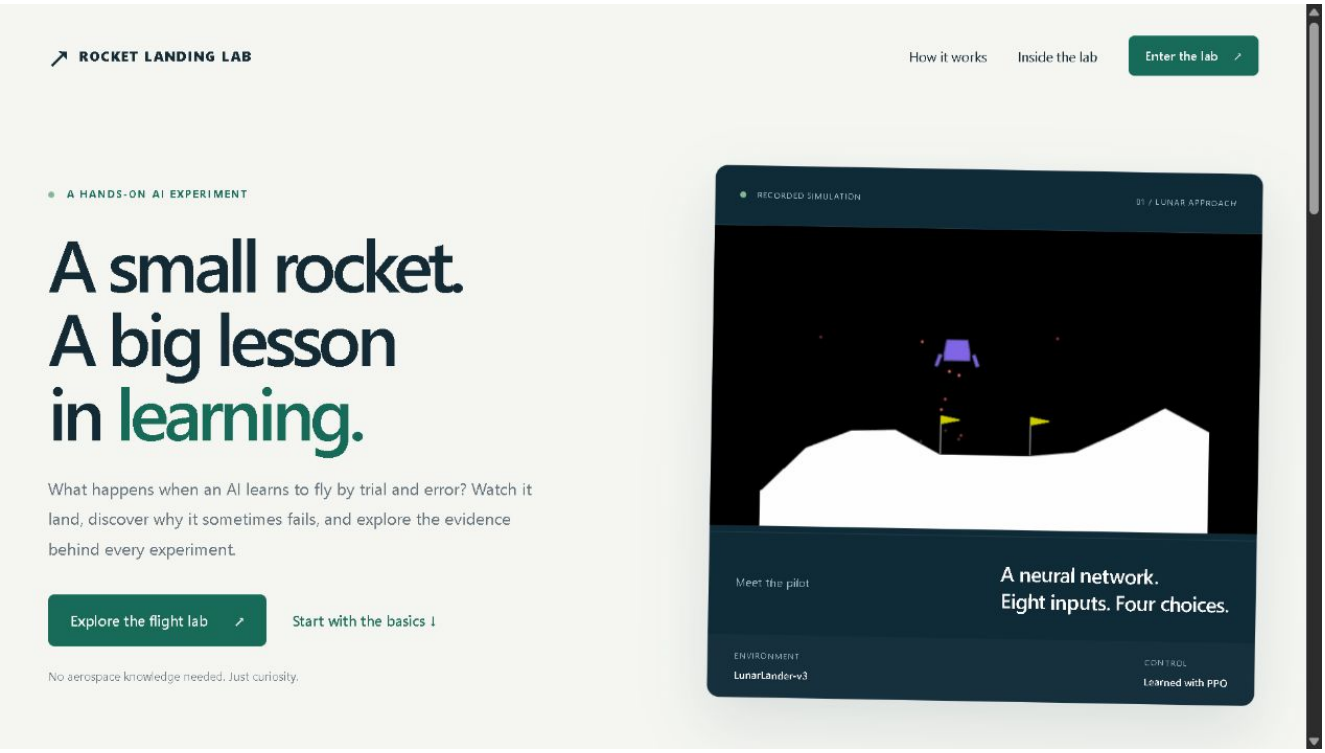


Figure 4 Guided introduction before the analytics and experiment workspace.

1. Double-click `Open-Web-Lab.cmd`. Keep its console window open while using the app. The launcher chooses an available local port and opens your browser. Close the console to stop the server.

2. Start on the introduction: explain the eight observations, four discrete actions, feedback through rewards, and the deliberately flawed hovering policy.

3. Enter the lab. Overview compares saved policies. Filter by weather and inspect the number of flights, landing rate, original score, and simulated duration. Export the filtered records as CSV.

4. Open Flight explorer. Search for `hover`, `failed`, a weather condition, or a seed. Play or scrub the reconstructed trajectory and inspect speed, tilt, and engine actions. The drawing is schematic, not a new video rendered by Box2D.

5. Open Failure predictor. Explain why a classifier is a separate model from the PPO controller. Inspect validation selection, held-out results, and the confusion matrix before making claims.

6. Ask the lab: "Why did the hover policy fail?" Then ask "Which pilot has the best landing rate in gusty weather?" Expand the evidence. The first is retrieval plus local generation; the second is a deterministic calculation over saved flights.

7. Open Broader training. Compare the source and fine-tuned policy under identical evaluation seed numbers. Additional training is optional; a completed comparison is included.

The older desktop control panel and launchers still work. Setup on another Windows computer: install Python (tested with 3.12), run `setup.ps1`, then open the web launcher. Internet is needed for dependency installation. Browser assets, simulations, analytics, and prediction run locally. Generated assistant answers additionally require Ollama with `qwen3:1.7b`; install that model using `ollama pull qwen3:1.7b` if desired. When it is unavailable, source retrieval still works and the UI explicitly labels excerpts instead of generated answers. No paid API key is required. This app serves only loopback; it is not configured for public multi-user hosting or GitHub Pages.

28 How the pieces connect

Additional web and ML dependencies

Component	Purpose
FastAPI 0.133.1 and Uvicorn 0.41.0	Local API and web server
HTML, CSS and JavaScript	Introduction, analytics and replay interface
SQLite and zlib	Embedded storage and compressed telemetry
scikit-learn 1.9.0 and Joblib 1.5.3	Classification, retrieval, metrics and model persistence
pypdf 6.10.2	Placement PDF extraction for retrieval
Ollama / Qwen3 1.7B	Local language generation
HTTPX 0.28.1	API test client

```
Gymnasium -> PPO action -> physics step -> FlightMetrics -> telemetry SQLite -> dashboard / outcome classifier
```

```
Question -> exact numerical query OR TF-IDF retrieval -> source excerpts -> local Ollama generation -> answer with source IDs
```

```
Saved fixed PPO -> randomized-weather fine-tuning -> separate checkpoint -> paired weather evaluation -> comparison
```

The frontend is HTML, CSS and browser JavaScript. FastAPI serves the page, assets and JSON endpoints. Uvicorn runs the HTTP server. SQLite is embedded storage, so there is no database service to configure. NumPy provides feature calculations. Scikit-learn supplies preprocessing, classifiers, metrics and text retrieval. Joblib stores the fitted classifier. PyPDF reads the placement PDF for retrieval. Gymnasium/Box2D supplies the environment; Stable-Baselines3/PyTorch supplies PPO training and inference. Matplotlib remains responsible for the original experiment reports. Python subprocesses run long jobs without blocking page requests. One job at a time is allowed; its log and stop button are visible.

29 Telemetry: what is actually stored

The included library contains 720 flights: four original policies x three weather conditions x 60 episode seeds, 600000-600059. All use deterministic policy actions and the original environment reward, with reward shaping disabled during evaluation. Calm, breezy and gusty use the same settings as the original weather tests. Each policy currently represents one original training seed.

`artifacts/lab/flights.sqlite` contains one row per flight. Summary columns: a flight ID, policy label, weather, episode seed, SHA-256 checkpoint identity, UTC collection time, outcome, original return, number of steps, fuel proxy and hover fraction. A compressed JSON blob stores the full sequence: step, x, y, horizontal speed, vertical speed, angle, angular speed, left contact, right contact, action and original step reward. Step zero is the reset observation; its action is -1 and reward is zero. Later rows record the observation after their action. Observations use Gymnasium scaling; they are not all SI units. One step represents 1/50 simulated second; wall-clock playback and training speed are separate.

The flight ID hashes checkpoint identity, scenario, episode seed and schema version. Recollection skips existing identical flights. A changed checkpoint produces new IDs. "Collect more flights" evaluates available final checkpoints on the configured default seed set; it can add a newly trained policy, but does not duplicate existing flights. For fresh seed numbers, use the telemetry command with `--seed` and `--episodes`. New collected data does not silently retrain the predictor: use its explicit training button.

Fuel is an engine-use proxy, not kilograms. `failed` includes crash/out-of-bounds termination. `timeout` means reaching the step cap. A high score is not equivalent to a successful landing. Filtering outcomes conditions the statistics: selecting only landed flights necessarily displays 100% landing rate. Compare policies with the same weather, seed set, and outcome filter. The 95% Wilson interval estimates uncertainty in a binomial landing proportion, not variation across training seeds. Shared seed numbers help comparisons but weather changes can alter random draws and trajectories.

30 Failure predictor: supervised learning without looking into the future

The target is one of `failed`, `landed`, `timeout`. The forecast is made at step 150, or three seconds. Flights ending by that point are excluded: their outcome is already known. There are 714 eligible flights and six excluded early endings in the included dataset.

For x, y, vx, vy, angle and angular speed, compute last value, mean, standard deviation, minimum and maximum over observations 0-150. Add main-engine and side-engine fractions from actions 1-150. This produces 32 features. The input deliberately excludes final return, final length, weather label, policy name, final outcome and observations after step 150. The final outcome is used only as the supervised target.

Split by episode seed group, keeping every policy/weather version of the same seed in one partition. GroupShuffleSplit first holds out 20% of groups for testing, then 25% of remaining groups for validation (approximately 60/20/20 overall). Fit a standard-scaled logistic regression and a random forest on training data only. Choose the model with the highest validation macro F1. Do not refit or select using the test set. The included selected model is a random forest; all split seed lists and per-flight assignments are saved.

Held-out results: accuracy 77.78%, macro F1 0.668, landing Brier score 0.162 on 144 flights. The confusion matrix has actual rows and predicted columns in the order failed/landed/timeout:

Actual	Predicted failed	Predicted landed	Predicted timeout
Failed	5	18	0
Landed	4	73	5
Timeout	0	5	34

This exposes a significant limitation: failure recall is only 5/23 = 21.7%. Accuracy alone conceals it. The predictor is an educational diagnostic, not a reliable crash alarm. Macro F1 weighs the three classes equally; Brier score measures squared probability error for landing, with lower being better. The reliability table compares predicted probability and observed landing frequency. Probabilities are not guaranteed to be calibrated. No threshold was tuned on the test data. Replay labels each flight as train, validation, test or new; an individual training-flight prediction is not independent evidence of performance.

Generalization is to unseen seed groups for the known policies, not unseen algorithms or real rockets. Adding policies, changing feature definitions or repeatedly consulting the test set requires a new evaluation protocol. A policy-held-out split and additional training seeds are useful future research. Model file hashing detects accidental inconsistency with the report; it does not authenticate untrusted files. Only load the repository's locally produced Joblib artifacts.

31 RAG: retrieval augmented generation

The assistant reads project documentation, PDF pages, Python source, and original quality-gate facts. It splits text into overlapping 210-word windows with a 150-word stride. Each chunk retains its file/page and source ID. TF-IDF with word unigrams and bigrams builds a sparse index. The question is transformed using the same vectorizer; cosine-like normalized dot products rank chunks. Up to four chunks with similarity above 0.055 become context.

This is lexical retrieval, not neural embeddings or a vector database. That is a valid, small local RAG baseline, but synonyms and complex questions may retrieve weak evidence. Ollama serves the local Qwen3 1.7B model. The prompt asks it to use supplied evidence, admit insufficient support, and cite source IDs. The app shows retrieved excerpts and file/page identifiers so users can inspect the grounding. Invalid source IDs are removed, but supported citations do not prove every generated sentence is correct. Generation failures fall back to labeled excerpts. No evidence produces an explicit "could not find supporting project evidence" answer.

Recognized numerical questions such as landing rate, average score, best pilot and number of flights use deterministic queries and arithmetic over the telemetry library. They are labeled "computed data," not passed to the language model to invent numbers. "Best" defaults to observed landing rate unless average score is requested; it is not a universal ranking. The assistant does not control the rocket, run arbitrary commands, or modify training from its text response. It indexes local material in memory on first use; restart the server after editing indexed documentation.

32 Broader RL training and honest comparison

The experiment starts from a copy of the fixed-reward policy. Each episode independently samples wind strength uniformly from 0-18 and turbulence from 0-1.8, with wind enabled. This is domain randomization: vary environmental conditions during practice. Four vectorized environments feed PPO. It continues learning with the original reward only, preserving the source checkpoint. The explicit new training seed controls Python/NumPy/PyTorch and environment seeds after loading the source model.

PPO collects full rollout batches, so 200,000 requested steps produce 200,704 additional steps with this configuration. Source and additional step counts are recorded separately. The final checkpoint goes into a new `runs/robust/seed_N` folder; `config.json` records source hash, weather ranges, reward and seed. Interrupted training folders may lack a final checkpoint and are not collected as policies.

After training, both source and new checkpoint are tested for 50 episodes in each of calm, breezy and gusty weather, seed numbers 700000-700049. The results page reports original return, landing rate and Wilson intervals. This is an exploratory comparison of two saved policies. Continued learning can improve one condition and degrade another. It also changes both weather exposure and reward relative to the source's original training, so improvements cannot be attributed solely to weather randomization. Strong causal evidence requires controlled ablations, multiple training seeds and a final untouched test set. Reusing these evaluation seeds for iterative tuning turns them into development data.

The corrected seeded run (seed 2, 200,704 additional steps) produced the following 50-episode rates: source vs new policy was 90% vs 92% calm, 92% vs 88% breezy, and 68% vs 76% gusty. Gusty mean original score increased from 141.6 to 159.4, while breezy declined from 217.0 to 201.5. These are observed differences with overlapping uncertainty intervals, not proof of a general improvement. An initial implementation check revealed a seed-reset issue and was archived; the corrected run was not chosen by ranking training runs. The evaluation seed set is therefore exploratory evidence, not a pristine final benchmark.

33 Files, endpoints and reproducibility

- `telemetry.py`: collection, compression, SQLite queries and episode identity.
- `predictor.py`: prefix features, grouped splits, candidate fitting, metrics and inference.
- `assistant.py`: document chunks, retrieval, exact data answers and local generation.
- `broader_training.py`: weather wrapper, fine-tuning and evaluation.
- `webapp.py`: API, local server and owned background jobs.
- `web/`: introductory page and five-tab interface.
- `artifacts/lab/`: curated telemetry, classifier, classifier report and latest broader comparison.

GET `/api/flights` lists summaries; `/api/flights/{id}` returns telemetry and forecast; `/api/summary` aggregates filters; `/api/export` exports CSV; `/api/predictor` and `/api/training` return measured reports. POST `/api/ask` handles questions. `/api/jobs` starts and inspects tasks; `/api/jobs/stop` stops the owned task. Jobs use fixed module names and validated settings, not shell commands from input. The server binds to localhost and rejects cross-origin POST requests. This is a local single-user application, not an authenticated public service.

Run `python -m pytest -q` in the project environment. New regression coverage checks telemetry round trips and idempotency, prefix-only features, split isolation, outcome exclusions, weather reproducibility, API filtering/export/input validation, and retrieval/data-answer behavior. Browser verification covers introduction, navigation, charts, replay, predictor, assistant, and training results. Saved checkpoints and classifier reports retain checksums. Dependency versions and the evaluation protocol still matter: seeded runs need not be bit-identical across platforms or library releases.

34 Placement explanation and interview prompts

"I built an end-to-end local reinforcement-learning experiment and analysis application. PPO learns control from reward; a separate supervised classifier forecasts outcomes from early telemetry. SQLite stores reproducible flight histories, a dashboard compares observed reliability, and a local RAG assistant explains project evidence. I evaluated models with independent seed groups and exposed weaknesses such as low failure recall instead of reporting accuracy alone."

Be clear that implementation was developed with AI assistance. Be prepared to explain each design decision and reproduce a small experiment yourself.

Questions to prepare: Why is PPO reinforcement learning while the failure predictor is supervised learning? What is reward hacking? Why use original rewards during evaluation? What information would cause target leakage? Why group by seed? Why does macro F1 differ from accuracy? What does a Wilson interval cover? What does a Brier score measure? What can TF-IDF miss? Why is RAG not fine-tuning? Can a cited answer still hallucinate? Why does randomized weather not guarantee robustness? What additional experiment would justify calling a model better?

Interest mapping: AI/ML covers control and classification; deep learning covers the PPO neural policy and local language model; RAG covers retrieval and grounded explanations; data analytics covers telemetry, filtering, exports, uncertainty and comparison. These components have separate responsibilities and measurable limits.

Practice tasks before placement

1. Alter only observations after step 150 and confirm the feature vector is unchanged.

2. Filter the included library to gusty conditions, confirm 240 flights, and compare the CSV rows with the dashboard.

3. Find a failed test flight whose prediction favors landing; connect it to the confusion matrix.

4. Stop Ollama and confirm a conceptual question falls back to labeled evidence excerpts.

5. Explain why the new model's breezy decline must be reported alongside the gusty gain.

6. Trace an assistant numerical answer to its seed range and distinguish it from the original calm benchmark.

7. Give a 60-second pitch that names all three learning systems and one limitation for each.

35 Technical deep dive and upgrade interview answers

Three models with separate jobs

System	Input and output	Method
PPO controller	Eight observations to an engine action	Actor-critic reinforcement learning
Outcome classifier	32 early-flight features to three probabilities	Supervised learning from final outcome labels
RAG assistant	Question plus evidence to an explanation	Lexical retrieval and pretrained local language generation

The project does not train Qwen. RAG supplies context at inference time. The classifier does not update or override PPO; it is diagnostic. Collecting telemetry evaluates saved weights without training them. These boundaries are important in both the architecture and an interview explanation.

Feature mathematics and leakage

For one signal observed at n=151 states (reset plus 150 transitions), mean = sum(x_i)/n. Population standard deviation = sqrt(sum((x_i-mean)^2)/n). NumPy std uses the population denominator by default. Latest value, minimum and maximum complete five statistics for each of six signals: 5 x 6 = 30 features. Two engine-use fractions produce 32 features.

If the main engine fired in 60 of the first 150 actions, main_fraction = 60/150 = 0.4. The reset action -1 is excluded. Final reward, final episode length and post-step-150 observations never enter the vector. The final outcome is the target, not an input. A regression test changes the future observations and verifies that the prefix features remain identical.

All policy/weather cases sharing one seed stay in the same partition. Otherwise correlated initial conditions could appear in both training and test data. This reduces one leakage source, but does not establish generalization to unseen policies. A policy-held-out experiment would answer a different question.

Candidate models and selection

Logistic regression standardizes features using training statistics, learns linear class scores and converts them to probabilities. A scikit-learn pipeline ensures preprocessing is fitted only on training data. Fitting a scaler before splitting would leak information about the test distribution even without labels. max_iter is 3,000 and random_state is 42.

The random forest contains 180 decision trees, maximum depth nine, and at least four examples per leaf. Trees split features to reduce class impurity; averaging their predictions captures nonlinear interactions. Depth and leaf constraints limit complexity but cannot guarantee generalization. Both candidates fit training data only. Validation macro F1 selects the forest, with no test-based choice and no refit afterward. This is a two-candidate baseline comparison, not exhaustive hyperparameter optimization.

Calculate the metrics yourself

Precision = TP/(TP+FP), recall = TP/(TP+FN), and F1 = 2*precision*recall/(precision+recall). Macro F1 averages each class equally. Accuracy counts all correct predictions divided by test examples. They emphasize different errors.

For failed flights, TP=5, FN=18 and FP=4. Precision is 5/9 = 55.6%; recall is 5/23 = 21.7%; F1 is 10/32 = 0.3125. Overall accuracy is (5+73+34)/144 = 77.8%. Predicting landing for 18 failed flights is the principal weakness: a good overall accuracy headline does not make this a dependable crash alarm.

Landing Brier score averages (p_i-y_i)^2, where p_i is predicted landing probability and y_i is the binary landing result. A 0.8 landing prediction followed by failure contributes 0.64; a landing contributes 0.04. The measured score is 0.162, not 83.8% accuracy. Reliability bins compare forecast probabilities with empirical frequencies; small sample sizes and one grouped split limit confidence.

Retrieval versus training a language model

TF-IDF emphasizes terms common in a chunk but less common across the corpus. Normalized sparse vectors permit query/chunk ranking through dot products. Word bigrams preserve phrases such as reward shaping. This inexpensive retrieval baseline is weaker on synonyms and questions that need evidence spread across many chunks. Neural embeddings are a possible extension, not a feature currently implemented.

RAG retrieves evidence and adds it to the prompt; fine-tuning changes model parameters. A vector database alone is not RAG. A generated answer with a citation can still misinterpret its source. The current assistant has smoke tests and retrieval/fallback tests, but no formal large-scale answer-quality benchmark. Inspect excerpts rather than equating traceability with factual correctness. Recognized numerical questions use exact database arithmetic instead of LLM-generated measurements.

Fair interpretation of broader training

The new model inherits 1,003,520 source training steps and adds 200,704 steps, totaling 1,204,224. Comparing it against a fresh model trained for only 200,000 total steps would be misleading. Weather randomization stays inside the same simulator family; this is not real-world transfer.

The experiment also changes reward from the source's fixed shaping to original reward, so it cannot isolate weather exposure as the sole cause. A stronger experiment would compare continued original-reward training with and without randomized weather under matched budgets and multiple seeds. The observed gusty gain has overlapping uncertainty intervals and is exploratory. Repeatedly tuning to these evaluation seeds would turn them into development data.

Interview questions and answers

Why SQLite? It provides embedded storage and transactions without a separate database service. This fits the local single-user app. Large concurrent deployments would need a separate service and storage design.

Why exclude early endings? By the three-second forecast horizon their outcomes are already known. Counting them as early predictions would inflate performance by predicting events that already happened.

Can class weights improve failure recall? Possibly. Class weighting, thresholds, richer prefix features and additional failure examples are experiments to select on training/validation data. Use fresh final test seed groups after repeated development rather than optimizing against the published matrix.

Why hash model files? Filenames can stay unchanged while bytes change. A hash connects a result to a checkpoint and detects inconsistencies; it is not proof against coordinated tampering.

Does training happen in the browser? No. JavaScript sends a validated request to FastAPI. A Python subprocess runs training; a reader thread captures progress while the browser polls job state. The frontend remains responsive.

Does the language model control the rocket? No. PPO controls actions, the classifier forecasts outcomes, and the assistant explains project evidence. It does not execute commands from generated text.

36 Conclusions and references

The project succeeds as a complete learning and evaluation demonstration: it trains controllable policies, shows a concrete reward-design failure, measures original-task outcomes, tests changed dynamics and presents evidence through a usable interface. The main lesson is that a learning objective, an observed score and the real task are related but distinct. Good engineering keeps them separate long enough to detect when they disagree.

For placement preparation, prioritize understanding the reward and evaluation paths, explaining PPO at an appropriate level, and defending the limits of the experiments. A technically honest discussion of failures and uncertainty is stronger than an unsupported claim that the model is universally reliable.

Primary references

[1] Farama Foundation. Gymnasium Lunar Lander documentation. Environment interface, observations, actions and wind arguments. https://gymnasium.farama.org/environments/box2d/lunar_lander/ . Accessed 19 October 2026.

[2] Schulman, Wolski, Dhariwal, Radford and Klimov. Proximal Policy Optimization Algorithms. 2017. https://arxiv.org/abs/1707.06347 . Algorithm reference; the application uses the Stable-Baselines3 implementation.

[3] Stable-Baselines3. PPO documentation. https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html . Accessed 19 October 2026. Current online documentation can differ from the saved installed version.

[4] Stable-Baselines3. Reinforcement Learning Tips and Tricks. https://stable-baselines3.readthedocs.io/en/master/guide/rl_tips.html . Evaluation guidance; independent test environments and repeatability are relevant to this project.

[5] Ng, Harada and Russell. Policy Invariance under Reward Transformations Theory and Application to Reward Shaping. 1999. Primary reward-shaping reference. https://people.eecs.berkeley.edu/~russell/papers/icml99-shaping.pdf . The project adds a separate time cost, so the complete preset is not claimed to preserve the original objective.

Local evidence and source map

The implementation details in this report are grounded in rocket_landing/rewards.py, training.py, models.py, evaluation.py, recording.py, scenarios.py, robustness.py, quality.py, catalog.py, jobs.py, panel.py, reporting.py and cli.py. Configuration and original results are in runs/<preset>/seed_0. The 600-flight suite is artifacts/quality/20261016-013658-498851. Tests are in tests/test_rewards.py, test_models.py, test_recording.py, test_upgrades.py and test_quality.py.

The learning and weather plots reproduced here are saved project artifacts, not illustrations of hypothetical performance. Numerical result tables are populated from the delivered JSON evidence during report generation. The project was inspired by Hari (@harixganesh)'s tutorial supplied by the user; no independent claim about unrelated robotics systems is needed to explain this implementation.

New source/evidence map: telemetry.py, predictor.py, assistant.py, broader_training.py, webapp.py and web/; artifacts/lab/flights.sqlite, predictor-report.json, predictor.joblib and broader-results.json; broader-evaluation/20261021-015237-393978/episodes.csv; runs/robust/seed_2/config.json and training.json; tests/test_web_lab.py. The results above are saved measurements, not illustrative targets.